Qwen Forecast

Do dado à previsão auditável

Guia didático da implementação · 5 de outubro de 2026

Este documento acompanha o código reorganizado deste projeto. Ele explica como transformar uma série de produção de água em um prompt, enviar esse prompt ao Qwen pelo Ollama e avaliar as previsões junto com Chronos e TimesFM.

> **O que o Qwen faz aqui?** Recebe números históricos como texto e gera os próximos sete valores. O projeto não treina nem ajusta os pesos do Qwen. A expressão *qwen_forecast* designa essa rotina de inferência com o modelo local **qwen3:1.7b**.

Os exemplos de código são extraídos dos arquivos reais. Caminhos são relativos à raiz do repositório. Imports aparecem apenas quando ajudam a entender uma dependência; o foco está em contratos, parâmetros, fluxo e prevenção de vazamento.

Roteiro de leitura

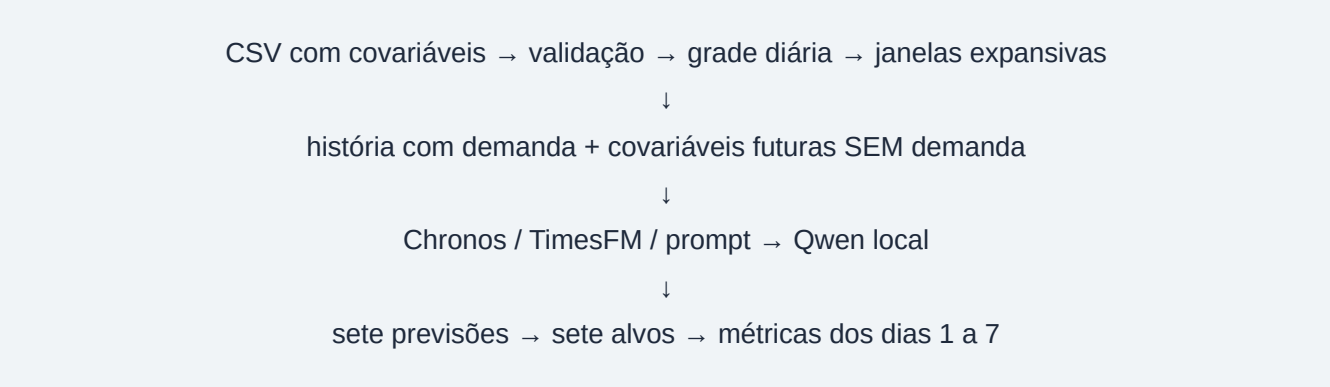
1. Arquitetura e protocolo temporal.
2. Leitura, validação e janelas.
3. Construção de prompts e exemplos few-shot.
4. Dataclasses, argumentos e interfaces.
5. HTTP, saída estruturada, validação e retentativas.
6. Orquestração, cache, métricas e resultados.
7. Como executar, testar e estender o projeto.

Leia primeiro os capítulos de prompts e Ollama se já conhecer validação temporal. Para reproduzir a execução, use o capítulo de comandos e a configuração em `configs/benchmark.json`.

1 · Arquitetura e covariáveis preservadas

A organização mantém dados, modelos e avaliação em camadas separadas. O contrato comum agora preserva as covariáveis dos arquivos originais. Nenhuma covariável configurada é descartada na leitura ou na construção do prompt.

Camada	Funções e responsabilidade
data/ingestion e validation	read_series lê o CSV; validate_series valida datas e demanda, conservando as demais colunas; write_json grava artefatos.
data/preprocessing	daily_grid explicita datas ausentes; fill_past_only preenche pelo passado; build_prompt monta o texto com demanda e covariáveis.
data/features/covariates.py	covariate_columns define o contrato; validate_covariates verifica valores; model_inputs separa história e covariáveis futuras; timesfm_covariates monta vetores.
evaluation	build_windows constrói janelas expansivas; deterministic_metrics calcula erros com todos os sete dias.
models	Chronos recebe future_df; TimesFM usa XReg + TimesFM; Qwen recebe tabelas de covariáveis no prompt.
pipelines	prepare_dataset prepara; run_dataset executa todas as janelas; compare confere alinhamento e emite dois CSVs.



O código concentra os nomes das covariáveis em `configs/benchmark.json`. Os três modelos recebem as mesmas colunas por cidade. O Qwen usa um histórico menor permitido no AGENTS.md, sem mudar as datas avaliadas.

2 · Janela expansiva e datas comuns

Dataset	Contexto inicial baselines	Crescimento	Qwen
Joinville	365 dias	365, 372, 379… 736	15 dias
Guaratuba	730 dias	730, 737, 744… 1451	15 dias

Para uma origem O, a história das baselines ocupa `[0, O)` e o alvo ocupa `[O, O+7)`. O início do histórico é sempre zero: ele não desliza nem descarta as observações iniciais. O passo é sete dias. Na segunda janela, acrescentam-se sete observações já conhecidas naquela nova origem.

O `AGENTS.md` atual permite até 15 dias para o PromptCast. Assim, Qwen recebe `[O-15, O)`, com as mesmas datas-alvo e as mesmas covariáveis futuras que os demais modelos. Essa exceção é explicitada no CSV de resultados.

Por que os primeiros 730 dias chegam a 31/12/2017?

O tratamento de grade diária da reorganização foi mantido. O CSV Guaratuba não contém 29/02/2016; esse dia é materializado e preenchido pelo passado. Os 730 dias de calendário terminam em 30/12/2017, e a primeira previsão começa em 31/12/2017. Contar somente linhas do CSV antigo produziria outra origem. Nenhum arquivo de entrada é alterado.

Alvos e covariáveis são informações diferentes

A demanda futura nunca é fornecida aos modelos. Chronos recebe um dataframe futuro contendo data, identificador e covariáveis; TimesFM recebe vetores de covariáveis para história e horizonte, mas somente demanda histórica; o prompt do Qwen separa as tabelas históricas e futuras.

As covariáveis meteorológicas futuras são mantidas como fornecidas no CSV, conforme solicitado. Sua disponibilidade na origem é uma hipótese de entrada não comprovada pelos arquivos. Se forem observações conhecidas apenas depois, o experimento é uma avaliação condicional a esses valores e não uma simulação operacional com previsões meteorológicas arquivadas.

Os pesos dos modelos ficam congelados. O TimesFM ajusta regressão em cada origem usando apenas alvos anteriores a ela. Uma extensão com validação deve reservar os últimos 20% do treino. Não há ajuste de hiperparâmetros sobre o teste nesta execução.

3 · Preparação que conserva as colunas

read_series e validate_series

`read_series(path)` lê o CSV e chama `validate_series(df)`. Agora a validação trabalha sobre `df.copy()`, preservando todas as colunas. Ela converte datas e demanda, ordena cronologicamente e rejeita datas duplicadas e valores inválidos. Não seleciona apenas demanda.

daily_grid e fill_past_only

`daily_grid` cria a grade diária e marca quais alvos foram realmente observados. `fill_past_only` preenche lacunas de demanda, identificadores e covariáveis com o último valor anterior. Não usa interpolação com valores futuros. A marca observed não é preenchida como se o alvo fosse real: janelas com alvos imputados são excluídas.

```
def fill_past_only(df: pd.DataFrame) -> pd.DataFrame:
    result = df.copy()
    columns = [c for c in result.columns if c not in {'timestamp', 'observed'}]
    result[columns] = result[columns].ffill()
    if result.water_produced.isna().any():
        raise ValueError('Cannot fill initial missing demand without past data')
    return result
```

validate_covariates(df, dataset)

Consulta as listas numéricas e categóricas da configuração. Falta de uma coluna, ausência de valor ou número não finito produz erro. Não faz seleção automática de features e não aprende estatísticas de normalização com os dados completos.

prepare_dataset

```
def prepare_dataset(path, context, *, horizon=7, step=7):
    df = fill_past_only(daily_grid(read_series(path)))
    windows = build_windows(df, context, horizon, step)
    return df, windows
```

O retorno é `(df, windows)`. O parâmetro context significa histórico inicial da janela expansiva. O asterisco exige nomes para horizon e step, evitando confundir parâmetros numéricos posicionais.

4 · @dataclass e a janela que cresce

`@dataclass` gera métodos como inicialização, representação e comparação. `frozen=True` impede reatribuir os campos depois da criação, tornando os limites de uma janela estáveis.

```
@dataclass(frozen=True)
class ForecastWindow:
    context_start: int
    origin: int
    horizon: int

    @property
    def history(self) -> slice:
        return slice(self.context_start, self.origin)

    @property
    def target(self) -> slice:
        return slice(self.origin, self.origin + self.horizon)
```

`@property` permite acessar `window.history` como atributo, embora esse acesso calcule um `slice`. O índice final é exclusivo: `slice(0,365)` contém 365 observações. `iloc` usa posições do dataframe.

```
def build_windows(df, context: int, horizon: int = 7, step: int = 7) -> list[ForecastWindow]:
    """Context is the initial history length. Every later history starts at zero."""
    if min(context, horizon, step) < 1:
        raise ValueError('Window parameters must be positive')
    windows = [ForecastWindow(0, origin, horizon)
               for origin in range(context, len(df) - horizon + 1, step)
               if df.iloc[origin:origin + horizon].observed.all()]
    if not windows:
        raise ValueError('No complete observed evaluation windows')
    return windows
```

A diferença decisiva é `ForecastWindow(0, origin, horizon)`. Ao aumentar origin de 365 para 372, a história passa de [0,365) para [0,372). Não se usa origin-context como início. O +1 em len(df)-horizon+1 admite a última janela completa.

`Demonstration` e `OllamaConfig` também são dataclasses. `asdict` converte campos em dicionário para persistência. Anotações de tipos documentam os contratos, mas não validam sozinhas os dados; as funções fazem as verificações explicitamente.

5 · Covariáveis compartilhadas no prompt

`model_inputs(df, window, dataset, history_days=None)` seleciona somente as colunas configuradas. Retorna história com demanda e covariáveis, e futuro sem demanda. Para as baselines, history_days=None mantém o início zero. Para Qwen, history_days=15 recorta somente a entrada histórica.

Dataset	Covariáveis numéricas	Categóricas
joinville	precipitation_median, temperature_2m_previsao_mean, shortwave_radiation_previsao_std, day, dayofweek_sin, dayofweek_cos, season_sin, season_cos	is_holiday
guaratuba	relative_humidity_mean, temperature_mean, radiation_mean, precipitation_mean, day, dayofweek_sin, dayofweek_cos, season_sin, season_cos	is_holiday_ctba_gtba_jve, is_weekend, is_carnival, is_school_recess_pr

build_prompt(history, horizon, *, demonstrations=None, future_covariates=None)

A função descreve produção diária em metros cúbicos, exige sete números não negativos em JSON, informa datas e demanda histórica e acrescenta duas tabelas CSV: covariáveis dos 15 dias anteriores e covariáveis das sete datas futuras. As datas futuras precisam coincidir exatamente com o horizonte solicitado.

`format_values` serializa demanda com duas casas decimais. `to_csv(index=False, float_format="%.6g")` representa covariáveis com seis algarismos significativos e omite o índice interno. O nome de cada coluna permanece explícito, permitindo relacionar clima, calendário e demanda.

Se future_covariates contiver water_produced, a função lança erro. Ela também rejeita datas incompatíveis e covariáveis ausentes na história. Alterar os alvos futuros não muda o prompt; há teste automatizado para essa propriedade.

```
history, future = model_inputs(df, window, dataset, history_days=15)
prompt = build_prompt(history, 7, future_covariates=future)
# future contém timestamp + covariáveis, nunca water_produced.
```

O prompt não chama HTTP. Essa tarefa pertence ao adaptador Qwen, separando preparação e inferência para permitir auditoria e teste.

6 · Exemplos few-shot e fronteira temporal

A comparação executada usa zero-shot, sem exemplos de respostas. As funções opcionais few-shot permanecem disponíveis no módulo de prompts, sem escolher estratégias a partir das métricas do teste.

Parâmetro de select_demonstrations	Função
training_values	Prefixo congelado anterior à primeira origem de teste.
history	Demanda da entrada atual; define largura e consulta de similaridade.
horizon	Quantidade de alvos em cada demonstração.
strategy	zero retorna lista vazia; few_random amostra; few_similar usa distância.
count e seed	Quantidade de exemplos e semente do gerador local.

Os candidatos têm entrada [o-width,o) e resposta [o,o+horizon), ambas inteiramente dentro do prefixo congelado. A amostragem aleatória não repete candidatos. A seleção por similaridade padroniza somente entradas e ordena sua distância quadrática; não consulta as respostas futuras dos candidatos.

As demonstrações opcionais atuais contêm pares de demanda histórica e resposta. As tabelas de covariáveis da tarefa corrente continuam no prompt, mas o benchmark principal não usa demonstrações. Um estudo few-shot com tabelas em cada exemplo seria uma extensão explícita, não uma mudança automática desta execução.

O orçamento do Ollama continua sendo validado. Se adicionar exemplos ultrapassar num_ctx, a inferência é recusada em vez de truncar silenciosamente a história.

training_partition

Para uma futura extensão de treino, `training_partition(first_test_origin, validation_fraction=0.2)` fornece cortes cronológicos dentro do prefixo. Não aplica fit e não recebe observações do horizonte da janela atual.

7 · args, *args, **kwargs e argumentos nomeados

São conceitos relacionados, mas diferentes. O objeto chamado `args` no programa vem do parser da linha de comando. Ele não é a mesma coisa que o padrão `*args` de uma assinatura Python.

```
parser.add_argument("--model", choices=["qwen", "chronos", "timesfm"], default="qwen")
args = parser.parse_args()
# args.model contém a escolha feita na linha de comando.
```

`parse_args()` devolve um `argparse.Namespace`, um objeto cujos atributos correspondem às opções registradas. `type=Path` converte um argumento textual em caminho; `choices` rejeita uma opção desconhecida antes de iniciar o modelo.

O asterisco sozinho

```
def build_prompt(history, horizon, *, demonstrations=None):
    ...

build_prompt(history, 7, demonstrations=examples)
```

O `*` sozinho não coleta argumentos. Ele exige que os argumentos seguintes sejam passados pelo nome. A chamada `build_prompt(history, 7, examples)` seria inválida. Isso torna uma opção adicional visível e reduz erros de ordem.

*args e **kwargs em uma definição

```
# Exemplo didático; não é necessário no pipeline atual.
def encaminhar(funcao, *args, **kwargs):
    return funcao(*args, **kwargs)

encaminhar(build_prompt, history, 7, demonstrations=examples)
```

Na definição, `*args` reúne argumentos posicionais extras em uma tupla. `**kwargs` reúne argumentos nomeados extras em um dicionário. Os nomes args e kwargs são convencionais: o que determina o comportamento são os asteriscos. Na chamada, os mesmos símbolos fazem a operação inversa, desempacotando tupla e dicionário.

** usado de verdade no projeto

```
OllamaConfig(**config["qwen"], seed=config["seed"])
# Equivale a passar model=..., base_url=..., temperature=..., etc.
```

As chaves do dicionário precisam coincidir com os nomes aceitos pelo construtor. Chave desconhecida ou argumento repetido gera `TypeError`. Não há função genérica que engula qualquer opção: o construtor tipado ajuda a detectar erros de configuração.

`dict(common, **deterministic_metrics(...))` também desempacota um dicionário, desta vez para compor uma linha do relatório. As anotações como `list[float]` explicam o contrato, mas não validam valores sozinhas: por isso há checagens explícitas no parser.

8 · OllamaConfig e a identidade do modelo

Ollama é o servidor local que carrega e executa o Qwen. Python atua como cliente HTTP. Não é preciso adicionar o SDK Ollama ao ambiente: a biblioteca padrão fornece a comunicação necessária.

Campo	Valor da execução	Efeito
model	qwen3:1.7b	Seleciona o modelo instalado.
base_url	http://127.0.0.1:11434	Endereço do servidor local.
temperature	0	Reduz aleatoriedade da geração; não garante reprodutibilidade bit a bit entre hardwares.
seed	42	Semente enviada ao runtime.
num_ctx	8192	Capacidade de contexto em tokens.
num_predict	256	Limite de tokens gerados, suficiente para sete números.
timeout	600 s	Prazo da operação HTTP; não é o tempo esperado por previsão.
retries	2	Até duas novas tentativas além da primeira.

`OllamaConfig.__post_init__` roda automaticamente após o inicializador gerado pela dataclass. Ele rejeita limites não positivos e quantidade negativa de retentativas. `frozen=True` evita que outra função mude uma configuração no meio da execução.

QwenForecaster.__init__

O construtor recebe uma instância de configuração e consulta `/api/tags`. Ele procura o nome exato e salva seu `digest`, além de `asdict(config)`, em `self.identity`. Se o modelo não está instalado, falha imediatamente. O programa não troca por outro modelo automaticamente.

`self` é a própria instância. Atributos como `self.config` e `self.last_audit` mantêm estado entre chamadas. Cada nova previsão substitui a auditoria anterior por uma estrutura referente àquela janela.

Interface comum

Os três adaptadores aceitam `predict(history, horizon, *, future_covariates=..., prompt=...)`; Chronos e TimesFM usam o dataframe futuro, e Qwen usa o prompt já construído com essas covariáveis. O pipeline consegue chamá-los do mesmo modo sem conhecer as APIs específicas. Isso é polimorfismo por contrato: não exige herança, desde que a assinatura e o resultado sejam compatíveis.

9 · HTTP e saída estruturada

post_json(url, payload, timeout)

```
def post_json(url: str, payload: dict, timeout: int) -> dict:
    request = urllib.request.Request(url, data=json.dumps(payload).encode('utf-8'),
                                     headers={'Content-Type': 'application/json'})
    with urllib.request.urlopen(request, timeout=timeout) as response:
        return json.load(response)
```

`json.dumps(payload)` transforma o dicionário em texto JSON; `encode("utf-8")` transforma o texto em bytes para o corpo HTTP. O cabeçalho informa o tipo do corpo. Ao fornecer `data`, a requisição usa POST. O bloco `with` fecha a resposta mesmo que ocorra uma exceção durante a leitura.

`json.load(response)` interpreta o envelope HTTP retornado pelo Ollama. O campo `response` dentro desse envelope ainda é uma string: ela contém o JSON que o modelo gerou e será validada separadamente.

O contrato enviado em format

```
def prediction_schema(horizon: int) -> dict:
    return {'type': 'object', 'properties': {'predictions': {
        'type': 'array', 'items': {'type': 'number', 'minimum': 0},
        'minItems': horizon, 'maxItems': horizon}},
        'required': ['predictions'], 'additionalProperties': False}
```

`minItems` e `maxItems` fixam o tamanho do vetor. `minimum=0` expressa a restrição física do consumo. `required` exige a chave, e `additionalProperties=False` impede chaves extras no contrato. A validação local repete as condições essenciais: não basta confiar no modelo ou no esquema.

Demais campos do payload

`stream=False` pede uma resposta completa, simplificando persistência e parsing. `think=False` desliga a saída de raciocínio do Qwen nesta configuração. `keep_alive="10m"` mantém o modelo carregado entre janelas. `options` contém temperatura, semente e limites; não é um argumento do sistema operacional.

Referências oficiais: Ollama /api/generate e structured outputs. Os exemplos deste manual descrevem a implementação local e não exigem serviços de nuvem.

10 · Validar números, limites e falhas

parse_prediction(text, horizon)

```
def parse_prediction(text: str, horizon: int) -> list[float]:
    """Reject prose, strings, booleans, NaN, missing values and wrong horizons."""
    parsed = json.loads(text)
    if not isinstance(parsed, dict) or set(parsed) != {'predictions'}:
        raise ValueError('Expected exactly the predictions key')
    values = parsed['predictions']
    if not isinstance(values, list) or len(values) != horizon:
        raise ValueError(f'Expected {horizon} predictions')
    if any(type(v) not in (int, float) or not math.isfinite(v) or v < 0 for v in values):
        raise ValueError('Predictions must be finite, nonnegative numbers')
    return [float(v) for v in values]
```

Usamos `type(v) not in (int, float)` intencionalmente: em Python, `bool` é uma subclasse de `int`. Uma validação com apenas `isinstance(True, int)` aceitaria `True` como previsão. Valores textuais como `"12"` também são recusados, evitando correções silenciosas.

`math.isfinite` recusa NaN e infinito mesmo que o decodificador JSON os aceite. O parser não remove cercas Markdown, não extrai números de explicações e não completa posições ausentes. Uma resposta inválida é registrada como falha, não substituída por uma previsão artificial.

QwenForecaster.predict

Antes do HTTP, conta os bytes UTF-8 do prompt e reserva `num_predict + 512` posições para saída e template. É uma estimativa conservadora para a tokenização do Qwen, que pode rejeitar um texto que caberia; o objetivo é impedir truncamento silencioso. A resposta também é verificada quanto a `done`, motivo `length` e contagens de tokens.

O laço executa no máximo `retries+1` tentativas. Cada uma registra resposta bruta ou erro. Em falhas previstas de JSON/validação, conexão ou timeout, espera 1 e depois 2 segundos antes de tentar de novo. Na última tentativa, `raise` propaga o erro original. A temperatura permanece a mesma: uma saída inválida determinística pode repetir-se.

O pipeline captura a falha da janela, grava `status="error"` e continua coletando diagnóstico. Ao final retorna código de saída não zero; o script de comparação não prossegue com resultados incompletos.

11 · Orquestração e rastreabilidade

main e create_model

`main` interpreta --model, --dataset, --config, --output e --limit. `create_model` importa somente a biblioteca necessária e passa a configuração do dataset aos adaptadores. Assim, a lista de covariáveis não fica escondida dentro de um modelo.

run_dataset: sequência de operações

1. Prepara os dados e valida todas as covariáveis configuradas.
2. Cria janelas expansivas desde o contexto inicial exigido.
3. Carrega o modelo e registra hashes e política de covariáveis.
4. Chama model_inputs para obter história e futuro sem demanda.
5. Para Qwen, gera o prompt com demanda e covariáveis e o salva.
6. Reaproveita somente cache compatível; caso contrário, executa predict.
7. Grava a resposta, tamanho real da história e auditoria.
8. Após inferência, acessa os sete alvos e gera sete linhas por janela.
9. Salva predictions.csv e status.json.

Para Chronos e TimesFM, o campo context em cada linha cresce a cada origem. Para Qwen ele é 15. O campo initial_context identifica a primeira origem comum de avaliação e não o tamanho do prompt.

fingerprint, source_fingerprint e write_json

Os hashes incluem dados, configuração, identidade do modelo, conteúdo dos módulos de src e texto do prompt. Alterações de protocolo invalidam o cache. write_json usa arquivo temporário e substituição atômica; não há gravação parcial no JSON principal.

O cache evita repetir respostas válidas após interrupções. Não execute duas instâncias escrevendo no mesmo diretório. A duração usa time.monotonic, e o modelo é carregado fora do cronômetro por janela.

O experimento anterior sem covariáveis foi separado em experiments/previous_demand_only. Ele não compõe os resultados atuais.

12 · Modelos e métricas dos sete dias

Chronos-2: covariáveis e história completa

`predict` mantém as covariáveis em context_df e em future_df. Acrescenta o identificador único da série e chama predict_df com prediction_length=7, batch_size=1, freq="D" e quantile_levels=[0.1,0.5,0.9]. A previsão pontual é a mediana 0.5. `context_length=len(frame)` impede que o tamanho inicial seja reutilizado como limite fixo.

TimesFM 2.5: XReg + TimesFM

Em cada chamada, recompila max_context como o próximo múltiplo de 32 do histórico atual. return_backcast=True habilita a regressão. `timesfm_covariates` concatena história e covariáveis futuras em vetores de tamanho contexto+7, separando numéricas e categóricas conforme a configuração.

`forecast_with_covariates(..., xreg_mode="xreg + timesfm")` ajusta a regressão com a demanda histórica e prevê resíduos com TimesFM. A demanda dos sete alvos não é passada ao ajuste. O resultado retornado já contém a combinação de regressão e modelo; não é necessário somar novamente.

Qwen: as mesmas covariáveis

Ollama recebe as covariáveis históricas dos 15 dias permitidos e as sete futuras no texto, além de demanda histórica. O esquema JSON exige sete previsões. Todas são persistidas e avaliadas.

RMSE agregado e RMSE por dia

```
# Agregado: todos os pares, dos dias 1 a 7 de todas as janelas.
rmse = np.sqrt(np.mean((y_pred - y_true) ** 2))
# Por antecedência: somente pares com lead == 1, depois 2, ..., 7.
```

São 378 pares por modelo em Joinville e 728 em Guaratuba. Não se calcula apenas ytrue0/yhat0 e ytrue1/yhat1. Também não se faz média dos RMSEs por janela. O CSV principal tem seis linhas, e o CSV por horizonte tem 42: duas cidades × três modelos × sete dias.

MAE é média do erro absoluto; MAPE é erro percentual absoluto com alvos zero excluídos e contados; sMAPE usa a soma dos módulos no denominador; R^2 compara a soma de erros quadráticos à dispersão dos alvos. compare exige datas/alvos idênticos, nenhuma janela perdida e crescimento correto do histórico.

13 · Resultados corrigidos — dias 1 a 7

Execução com covariáveis e histórico expansivo das baselines. O Qwen usa 15 dias, conforme permitido no AGENTS.md. A tabela principal agrega todos os sete dias previstos em cada janela.

Dataset	Modelo	MAE	RMSE 1–7	MAPE %	R^2
joinville	qwen	12325.544	15441.573	5.694	-0.127
joinville	chronos	6183.122	8036.899	2.813	0.695
joinville	timesfm	6034.872	8076.895	2.763	0.692
guaratuba	qwen	2143.011	2806.129	16.948	0.362
guaratuba	chronos	1137.943	1422.610	9.331	0.836
guaratuba	timesfm	1251.872	1576.313	10.201	0.799

RMSE separado por antecedência

Dataset	Modelo	Dia 1	Dia 2	Dia 3	Dia 4	Dia 5	Dia 6	Dia 7
joinville	qwen	12984.12	14549.04	15187.78	16298.58	16546.66	14973.94	17161.31
joinville	chronos	7736.69	6994.78	8259.14	8539.85	8469.72	6163.78	9617.07
joinville	timesfm	8210.95	7207.85	7922.48	9737.55	7955.81	6033.19	8944.33
guaratuba	qwen	2898.67	2723.22	2498.02	2693.60	2640.75	2893.43	3234.40
guaratuba	chronos	1406.26	1223.50	1223.22	1386.87	1462.20	1407.89	1775.49
guaratuba	timesfm	1570.16	1471.74	1502.66	1541.98	1495.87	1491.38	1914.26

O RMSE dos scripts antigos próximo de 1.280 era do primeiro dia. Ele não equivale ao RMSE agregado dos sete dias. Mesmo ao comparar o primeiro dia, é necessário considerar a configuração e as datas de teste: o protocolo atual segue os contextos iniciais 365/730, passo sete e a grade diária explicitada neste manual.

Dataset	Modelo	Primeiro contexto	Último contexto	Janelas
joinville	qwen	15	15	54
joinville	chronos	365	736	54
joinville	timesfm	365	736	54
guaratuba	qwen	15	15	104
guaratuba	chronos	730	1451	104
guaratuba	timesfm	730	1451	104

14 · Executar e conferir as entradas

```
# Em um terminal, se o Ollama ainda não estiver ativo:
ollama serve
# Na raiz do projeto, em outro terminal:
bash experiments/run_benchmark.sh

# Apenas Qwen:
conda run -n promptcast python -m src.pipelines.forecasting_pipeline --model qwen
# Métricas dos três modelos já concluídos:
conda run -n promptcast python -m src.pipelines.compare_models
```

Os ambientes existentes não são modificados. O script executa os modelos sequencialmente. A variável XLA_PYTHON_CLIENT_PREALLOCATE=false afeta somente a alocação de memória do processo TimesFM, não o protocolo de covariáveis.

Auditar crescimento e covariáveis

1. No predictions.csv do Chronos ou TimesFM, agrupe por origin e confira context: 365, 372, 379… em Joinville; 730, 737, 744… em Guaratuba.
2. history_start deve permanecer constante nas baselines.
3. No Qwen, confira context=15 e as mesmas datas-alvo.
4. Abra um prompt e procure todas as colunas numéricas/categóricas da configuração nas tabelas histórica e futura.
5. A tabela futura não pode conter water_produced.
6. Para cada origem, predictions.csv deve conter leads 1,2,3,4,5,6,7.

O uso de --limit 1 serve apenas para teste de integração e deve ter diretório separado. O comparador rejeita uma execução limitada como relatório final. Alterações de código/configuração invalidam o cache; use um novo diretório para preservar uma execução anterior.

15 · Testes e critérios de aceitação

A implementação passou por testes sem carregar modelos e por testes de inferência real. Os testes devem detectar regressões do comportamento solicitado, especialmente a perda de covariáveis e o uso acidental de janela fixa.

Propriedade	Como é verificada
Histórico expansivo	Início zero, tamanho inicial correto e incremento de sete nas primeiras janelas reais.
Chronos sem truncamento inicial	Adaptador simulado registra context_length igual ao tamanho completo em três origens.
Covariáveis preservadas	Conjunto de colunas é exatamente o configurado; vetores TimesFM têm história+7 posições.
Datas iguais para Qwen	Recorte curto muda apenas história, não as covariáveis futuras nem as datas-alvo.
Alvos futuros ausentes	Modificar water_produced futuro não muda o prompt; passá-lo no dataframe futuro gera erro.
Sete dias nas métricas	Erros somente nos dias 3 a 7 precisam afetar o RMSE agregado; nenhum lead pode ser omitido.
Validação de resposta	JSON com tamanho errado, booleanos, NaN, negativos ou texto é rejeitado.
Lacunas e validação	Preenchimento só pelo passado; alvo imputado não é avaliado; validação fica dentro do treino.

```
conda run -n promptcast python -m unittest discover -s tests -v
```

Na conferência final, as métricas são recalculadas com scikit-learn e os hashes dos arquivos de entrada são verificados. A presença de covariáveis meteorológicas futuras continua condicionada à hipótese de disponibilidade de entrada documentada, não inferida por um teste unitário.

Referências

- Ollama generate e saída estruturada.
- Chronos, biblioteca 2.3.1, modelo amazon/chronos-2.
- TimesFM, biblioteca instalada 2.0.2, pesos TimesFM 2.5 200M.
- Dataclasses e argparse.

Apêndice A · build_prompt completo

Fonte: `src/data/preprocessing/generate_prompts.py`. O corpo abaixo é extraído diretamente da implementação entregue.

```
def build_prompt(history: pd.DataFrame, horizon: int, *,
                 demonstrations: list[Demonstration] | None = None,
                 future_covariates: pd.DataFrame | None = None) -> str:
    """Accept history only, not the complete dataframe or a target dataframe."""
    if horizon < 1 or history.empty:
        raise ValueError('History and positive horizon are required')
    values = history.water_produced.to_numpy(dtype=float)
    if not np.isfinite(values).all():
        raise ValueError('History contains invalid values')
    dates = future_dates(history.timestamp.iloc[-1], horizon)
    parts = [
        'Forecast daily water production in cubic metres.',
        'Use the chronological daily history below to infer level, trend and seasonality.',
        f'Return ONLY a JSON object with key "predictions" and exactly {horizon} nonnegative numbers.',
        'Do not explain. Do not repeat the historical values as an answer.'
    ]
    for example in demonstrations or []:
        parts.append('Historical example input: ' + format_values(example.history))
        parts.append('Historical example output: ' + json.dumps({'predictions': list(example.target)}))
    parts.extend([
        f'History: {len(history)} consecutive daily observations, '
        f'from {history.timestamp.iloc[0].date()} to {history.timestamp.iloc[-1].date()}.',
        'Values (oldest to newest): ' + format_values(values),
        'Forecast dates in output order: ' + ','.join(dates.strftime('%Y-%m-%d')),
        f'Output schema: {{"predictions": [number, ...]}}; array length must be {horizon}.'
    ])
    if future_covariates is not None:
        if 'water_produced' in future_covariates.columns:
            raise ValueError('Future target must never appear in a prompt')
        if len(future_covariates) != horizon or not
dates.equals(pd.DatetimeIndex(future_covariates.timestamp)):
            raise ValueError('Future covariate dates must match the requested horizon')
        columns = [c for c in future_covariates.columns if c != 'timestamp']
        if not set(columns).issubset(history.columns):
            raise ValueError('Historical and future covariates must match')
        parts.extend([
            'Use the historical and supplied future covariates below in this forecast.',
            'Covariate values for forecast dates are supplied by the dataset.',
            'Historical covariates (CSV):',
            history[['timestamp'] + columns].to_csv(index=False, float_format='%.6g').strip(),
            'Future covariates (CSV, no future water production):',
            future_covariates.to_csv(index=False, float_format='%.6g').strip()
        ])
    return '\n'.join(parts)
```

Apêndice B · predict completo

Fonte: `src/models/promptcast.py`. A auditoria mantém cada tentativa e só retorna após validar a resposta.

```
def predict(self, history, horizon: int, *, prompt: str, future_covariates=None) -> list[float]:
        """Send only a prebuilt, target-free prompt; retain every attempted response."""
        config = self.config
        # UTF-8 byte count is a conservative upper bound for byte-level tokenizers.
        # Reserve space for the Ollama model template and the entire generated answer.
        if len(prompt.encode('utf-8')) + config.num_predict + 512 > config.num_ctx:
            raise ValueError('Prompt exceeds conservative context budget; increase num_ctx')
        payload = {'model': config.model, 'prompt': prompt, 'stream': False,
                   'think': False, 'keep_alive': '10m', 'format': prediction_schema(horizon),
                   'options': {'temperature': config.temperature, 'seed': config.seed,
                               'num_ctx': config.num_ctx, 'num_predict': config.num_predict}}
        self.last_audit = {'request': payload, 'attempts': []}
        for attempt in range(config.retries + 1):
            audit = {'attempt': attempt + 1}
            self.last_audit['attempts'].append(audit)
            try:
                answer = post_json(config.base_url + '/api/generate', payload, config.timeout)
                audit['response'] = answer
                if not answer.get('done') or answer.get('done_reason') == 'length':
                    raise ValueError('Incomplete or token-limited Ollama response')
                if answer.get('prompt_eval_count', 0) + answer.get('eval_count', 0) >= config.num_ctx:
                    raise ValueError('Context limit reached')
                return parse_prediction(answer.get('response', ''), horizon)
            except (ValueError, urllib.error.URLError, TimeoutError) as exc:
                audit['error'] = str(exc)
                if attempt == config.retries:
                    raise
                time.sleep(min(2 ** attempt, 4))
        raise RuntimeError('Unreachable retry state')
```